

2026 年 2 月 4 日

1 スミス標準形を用いた線形合同式解法コードの技術解説

本アルゴリズムは、整係数行列の対角化を通じて、多変数の線形合同式を独立した 1 変数の合同式に分解し、一般解を導出する。以下、各関数の役割と数学的背景を詳述する。

1.1 拡張ユークリッドの互除法 (gcd_extended)

関数 `gcd_extended(a, b)` は、単に最大公約数 $g = \gcd(a, b)$ を求めるだけでなく、ベズーの等式：

$$ax + by = g$$

を満たす整数解 (x, y) を再帰的に求める。これは後述する 1 次合同式の特殊解を構成するために不可欠なステップである。

1.2 1 次合同式のスカラ解法 (solve_linear_congruence_scalar)

合同式 $ax \equiv b \pmod{m}$ が解を持つための必要十分条件は、 $g = \gcd(a, m)$ が b を割り切ることである。コードでは以下の手順で特殊解を求めている。

- 拡張ユークリッドの互除法により $ax + my = g$ となる x を得る。
- 両辺に b/g を乗じることで、 $a(x \cdot \frac{b}{g}) + m(y \cdot \frac{b}{g}) = b$ を導く。
- これを法 m で考えることで、一つの解 $x_0 = (x \cdot \frac{b}{g}) \pmod{m}$ が得られる。

1.3 スミス標準形の計算アルゴリズム (SmithNormalForm クラス)

クラス `SmithNormalForm` は、行列 A を $S = UAV$ の形に変換する。ここで U, V は行列式が ± 1 の可逆な整係数行列（単因子行列）である。

1.3.1 ピボット選択と絶対値の最小化

コード内の `while True` ループの冒頭では、対象範囲内の非ゼロ成分のうち絶対値が最小のものを (k, k) 位置に移動させる。これは、ユークリッドの互除法における「法」を最小化し、掃き出

し回数を減らすためである。

1.3.2 ユークリッド的掃き出しと changed フラグ

行操作 `_add_row` および列操作 `_add_col` では、以下の剰余演算に相当する操作を行う。

$$A[r][k] \leftarrow A[r][k] - \lfloor A[r][k]/A[k][k] \rfloor \cdot A[k][k]$$

この操作後、成分が 0 にならない（割り切れない）場合、絶対値は必ず元のピボット $A[k][k]$ より小さくなる。このとき `changed = True` となり、より小さな値を新しいピボットとして再度掃き出しが行われる。この収束プロセスにより、最終的に第 k 行と第 k 列の非対角成分はすべて 0 となる。

1.4 体系の解法 (`solve_system_mod`)

線形合同式 $A\mathbf{x} \equiv \mathbf{b} \pmod{P}$ を解く際、変換行列を利用して以下のように変形する。

1. $S = UAV \implies A = U^{-1}SV^{-1}$ を代入し、 $U^{-1}SV^{-1}\mathbf{x} \equiv \mathbf{b} \pmod{P}$ 。
2. 両辺に U を乗じ、 $S(V^{-1}\mathbf{x}) \equiv U\mathbf{b} \pmod{P}$ 。
3. $\mathbf{z} = V^{-1}\mathbf{x}$ 、 $\mathbf{b}' = U\mathbf{b}$ と置くと、対角体系 $S\mathbf{z} \equiv \mathbf{b}' \pmod{P}$ が得られる。
4. 各 $s_i z_i \equiv b'_i \pmod{P}$ を独立に解き、最終的に $\mathbf{x} \equiv V\mathbf{z} \pmod{P}$ によって元の変数の解を復元する。

1.5 本実装の優位性

本コードは浮動小数点演算を一切含まず、整数の整除性のみに基づいている。そのため、法 P が大きな合成数である場合や、行列が特異（フルランクでない）に近い場合でも、数値的な不安定性を排除して厳密な一般解を計算できる点が、量子符号の設計における代数的な制約条件の探索に非常に適している。

2 行列の基本変形と変換行列 U, V の対応関係

スミス標準形 (SNF) の計算過程において、行列 A に対する操作が U および V にどのように反映されるかは、不変式 $S = UAV$ を維持するという観点から定義される。ここで、 U は行操作を記録し、 V は列操作（変数の変換）を記録する。

2.1 行操作と U の対応：左からの基本行列

行列 A に対する行操作は、数学的には左から基本行列 E を乗じることに相当する。初期状態を $I \cdot A \cdot I = A$ とすると、一連の行操作 $E_1, E_2, \dots, E_n$ は次のように累積される。

$$(E_n \dots E_1 I)A = UA$$

したがって、コード内での対応は以下のようになる。

- A の第 i 行と第 j 行を入れ替える: U の第 i 行と第 j 行を入れ替える。
- A の dst 行に src 行の k 倍を加える: U の dst 行に src 行の k 倍を加える。

2.2 列操作と V の対応：右からの基本行列と変数変換

行列 A に対する列操作は、右から基本行列 E を乗じることに相当する。 $S = UAV$ を維持するためには、列操作の履歴を V に蓄積していく必要がある。

$$UA(IE_1E_2 \dots E_m) = UAV$$

実装上の注意点として、 V に対する更新は**「列操作」ではなく「行操作」**として記述されることが多い。これは、線形方程式 $A\mathbf{x} = \mathbf{b}$ において、 $\mathbf{x} = V\mathbf{z}$ という変数変換を行っているためである。

- A の第 i 列と第 j 列を入れ替える: V の第 i 行と第 j 行を入れ替える。
- A の dst 列に src 列の k 倍を加える: V の src 行に dst 行の $-k$ 倍を加える（あるいは V 自体に右から基本行列を掛ける操作に相当する処理を行う）。

2.3 不変式の維持と計算の整合性

アルゴリズムの各ステップ終了時において、以下の不変式が常に成立している。

操作対象	数学的表現	変換行列の役割
行 (Row)	$A \leftarrow EA$	$U \leftarrow EU$ (U は行操作の積)
列 (Col)	$A \leftarrow AE$	$V \leftarrow VE$ (または $V^{-1} \leftarrow E^{-1}V^{-1}$)

最終的に A が対角行列 S に到達したとき、得られた U と V を用いて元の行列を $A = U^{-1}SV^{-1}$ と分解できる。これにより、法 P における合同式は次のように変形され、解が導出される。

$$A\mathbf{x} \equiv \mathbf{b} \pmod{P} \implies S(V^{-1}\mathbf{x}) \equiv U\mathbf{b} \pmod{P}$$

ここで $\mathbf{z} = V^{-1}\mathbf{x}$ と置くと、 $\mathbf{x} \equiv V\mathbf{z} \pmod{P}$ となり、対角体系の解 $\mathbf{z}$ から元の解 $\mathbf{x}$ を復元できる。